

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



FIUSAC
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Título del Práctica:

Modificación de funciones del kernel

PONDERACIÓN: 5 pts



Tiempo estimado: 4 hrs/min

Índice

1. MARCO FORMATIVO.....	3
1.1. Valor.....	3
1.2. Competencia(s).....	3
1.3. Habilidad(es) blandas a formar.....	3
2. Resultado del Aprendizaje.....	3
2.1. Objetivo SMART.....	3
3. Enunciado de la Práctica.....	4
3.1 Descripción del problema a resolver.....	4
3.2 Alcance de la práctica.....	4
4. Entregables.....	5
5. Material de apoyo.....	5
6. Recursos y herramientas a utilizar.....	5
7. Cronograma.....	6
8. Rúbrica de Calificación.....	6
8.1 Requisitos para optar a la calificación.....	6
8.2 Resumen de Puntuaciones.....	7
Detalle de la Calificación.....	8
Comentarios Generales.....	9

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Rigor y Responsabilidad Técnica	Dado que la modificación del kernel es una tarea crítica que puede comprometer la estabilidad de todo el sistema operativo, el estudiante debe trabajar con precisión, siguiendo protocolos estandarizados de desarrollo a bajo nivel para asegurar la integridad de la arquitectura.

1.2. Competencia(s)

Con la elaboración de esta práctica usted adquirirá la siguiente competencia:

El estudiante evalúa las capacidades, limitaciones y características de seguridad de distintos sistemas operativos mediante la identificación de sus fortalezas, vulnerabilidades conocidas y mecanismos de protección (SELinux, AppArmor, namespaces) para seleccionarlos y aplicarlos estratégicamente, considerando criterios de seguridad, en proyectos y tareas profesionales.

1.3. Habilidad(es) blandas a formar

La práctica le permitirá desarrollar las siguientes habilidades:

- Pensamiento Analítico: Capacidad para diseccionar la estructura del código fuente del kernel y entender el flujo de una llamada al sistema.
- Resolución de Problemas Complejos: Habilidad para diagnosticar y corregir errores de compilación o fallos de segmentación en un entorno privilegiado.
- Resiliencia y Perseverancia: Gestión de la tolerancia a la frustración ante procesos de compilación extensos o errores inesperados en el proceso de arranque del kernel modificado.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Modificar el código fuente del kernel de Linux para implementar un contador global dentro de la llamada al sistema <code>sys_getpid()</code> y recompilar el núcleo funcionalmente.	El éxito se determinará mediante la verificación de los registros del sistema (logs) o un programa de prueba en espacio de usuario que demuestre el incremento secuencial del contador tras cada invocación.	A través de la edición de los archivos de cabecera y archivos fuente del kernel (.c), la configuración de los parámetros de compilación y el uso de las herramientas de construcción (make).	Para que el estudiante comprenda de manera tangible la transición entre el espacio de usuario y el espacio de kernel, así como el impacto directo de la programación de sistemas en el comportamiento del hardware.	El desarrollo completo de la práctica, desde la modificación hasta la prueba final, debe completarse en un tiempo estimado de 3 horas de trabajo efectivo.

3. Enunciado de la Práctica

Esta práctica tiene como objetivo que los estudiantes comprendan la estructura interna del kernel de Linux y su proceso de compilación mediante la modificación de una llamada al sistema real. En este caso, se añadirá un contador que registre cuántas veces se ha invocado la función **`sys_getpid()`**, demostrando cómo los cambios en el código del kernel pueden alterar su comportamiento, y se implementará una llamada al sistema que imprima este contador llamada **`getpid_counter()`**.

3.1 Descripción del problema a resolver

En el desarrollo de software moderno, el sistema operativo actúa como una "caja negra" para la mayoría de los programadores. Sin embargo es fundamental comprender que cada acción realizada en el espacio de usuario (como obtener el ID de un proceso) depende de una ejecución privilegiada en el Kernel.

El reto planteado consiste en la instrumentación del núcleo de Linux. Actualmente, la llamada al sistema `sys_getpid()` simplemente devuelve el PID del proceso actual sin dejar

rastreo de su frecuencia de uso. El estudiante deberá intervenir el código fuente del kernel para implementar un mecanismo de monitoreo interno que registre de forma acumulativa cada invocación a esta función. El problema no solo requiere programar la lógica del contador, sino asegurar que dicha modificación no comprometa la estabilidad del sistema y que los resultados sean visibles para el administrador a través de los buffers de mensaje del sistema (dmesg).

3.2 Alcance de la práctica

La práctica se considera completada con éxito cuando se cumplen lo siguiente:

1. Obtención y Preparación: Descarga y extracción del código fuente de una versión estable del Kernel de Linux (versión indicada en el lab).
2. Modificación del Código: Localización exacta del archivo fuente donde reside `sys_getpid()` y la declaración de una variable global para el conteo. A su vez la creación de una nueva llamada de sistema para visualizar este contador en `dmesg`.
3. Gestión de Logs: Implementación de la función `printk` para imprimir en el log del kernel el valor actual del contador.
4. Compilación e Instalación: Generación del binario del nuevo kernel , instalación de los módulos necesarios y actualización del gestor de arranque (GRUB).
5. Verificación: Ejecución de un programa en C en espacio de usuario que invoque `sys_getpid()` repetidas veces y la posterior comprobación de que el contador incrementa correctamente en los registros del sistema llamando a `getpid_counter()`.

3.3 Requerimientos técnicos

Para el desarrollo de esta práctica, el estudiante requerirá un entorno de desarrollo controlado para evitar la pérdida de datos en su máquina anfitriona.

3.3.1 Requerimientos de Hardware (Mínimos sugeridos para VM)

- Procesador: Mínimo 2 núcleos físicos asignados a la VM (para acelerar la compilación).
- Memoria RAM: 4 GB (la compilación del kernel es intensiva en memoria).
- Almacenamiento: 30 GB de espacio libre en disco.

3.3.2 Requerimientos de Software

- Sistema Operativo: Distribución Linux (basado en Debian).
- Virtualizador: VMware Workstation, VirtualBox, VirtManager, QEMU, etc.

- Herramientas de Compilación:
 - build-essential: Compilador GCC y utilidades de enlace.
 - libncurses-dev: Para la interfaz de configuración del kernel (menuconfig).
 - flex y bison: Generadores de analizadores sintácticos.
 - libssl-dev y libelf-dev: Bibliotecas necesarias para el cifrado y manejo de archivos objeto del kernel.

3.3.3 Código Fuente

Kernel: Fuente oficial disponible en kernel.org.

4. Entregables

Describe los productos concretos que se espera que los estudiantes entreguen al finalizar la práctica. Pueden ser prototipos, informes técnicos, documentación, etc.

Tipo	Descripción
Informe Técnico	Documento en formato Markdown dentro del repositorio, que debe incluir: 1. Introducción y objetivos. 2. Guía paso a paso de los archivos modificados en el código fuente del kernel. 3. Capturas de pantalla de la configuración (menuconfig) y del proceso de compilación. 4. Evidencia de la validación mediante el comando dmesg. 5. Conclusiones y lecciones aprendidas.
Enlace Repositorio con Código Fuente	1. Los archivos específicos del kernel que fueron intervenidos (ej. syscall_64.tbl). <i>Este debe de seguir la misma estructura de carpetas del kernel, subiendo únicamente los archivos modificados (No subir todo el kernel a repositorio).</i> 2. El código fuente del programa en C (espacio de usuario) utilizado para llamar a getpid() y getpid_counter(). <i>Este código debe ir dentro de la carpeta Programa_Intermedio.</i> Todo debe de ir dentro de la carpeta Practica_1_2S2026 dentro de su rama (nombre de rama será su carnet).

5. Material de apoyo

- Love, R. (2010). Linux Kernel Development (3rd Edition). Addison-Wesley.
- Corbet, J., Rubini, A., & Kroah-Hartman, G. (2005). Linux Device Drivers (3rd Edition). O'Reilly Media.
- Documentación oficial del kernel Linux: <https://www.kernel.org/doc/html/latest/>

6. Recursos y herramientas a utilizar

Listado de materiales que los estudiantes deberán usar o investigar:

- **Software/Hardware:**
 - gnu/linux (distribución basada en Debian).
 - Herramientas de compilación (build-essential, make, gcc).
 - Editor de texto (recomendado: Visual Studio Code, Vim o Nano)
- **Plataformas:**
 - GitLab (para repositorio y control de versiones).
 - UEDI o Classroom (para entrega oficial).
- **Lecturas recomendadas:**
 - Documentacion del kernel [kernel.org](https://www.kernel.org/doc/html/latest/)
 - [Introducción a la documentación del kernel linux](#)
 - [About Dmesg in linux](#)

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Práctica	07/08/2026	07/08/2026
Elaboración	07/08/2026	19/08/2026
Calificación	20/08/2026	22/08/2026

8. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Antes de la evaluación de la práctica, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

Tema	Descripción	Cumple (Sí/No)
Cumplimiento de la tecnología establecida	La práctica debe desarrollarse en un sistema GNU/Linux basado en Debian (Linux Mint, Ubuntu, Debian o derivados), ya sea en máquina virtual o instalación nativa.	
Integridad de la entrega	Se debe entregar el enlace a la rama del repositorio GitLab que contenga el código completo y la documentación detallada de la práctica.	
Compilación Funcional	El kernel modificado debe ser capaz de arrancar (boot). Si el sistema no inicia debido a errores de configuración o compilación, no se procederá con la revisión técnica.	

8.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Habilidades		
Modificación de syscall <code>get_pid()</code>	25	
Implementación de la syscall <code>getpid_counter()</code>	30	
Validación con programa de prueba	20	
Uso correcto de <code>printk</code> y niveles de Log	5	
Uso de comando <code>dmesg</code> para verificación de syscall	10	

Sub-Total Habilidades	90	
2. Conocimiento		
Informe Técnico	5	
Pregunta Teórica	5	
Sub-Total Conocimiento	10	
TOTAL	100	

Detalle de la Calificación

En este apartado se deberá usar la siguiente plantilla para la rúbrica que debe tener obligatoriamente el criterio de evaluación, el rango de calificación **Satisfactorio (100%–61%)** y el rango de necesita mejorar **(60%–0%)**.

	Criterio/Nivel			Nivel de evaluación		
No.	Criterio de evaluación	Punteo maximo		Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
1	Habilidades					
1.1	Modificación de syscall get_pid()	25	Descripción	Se logró modificar el archivo correcto y la syscall get_pid() correctamente, logrando compilar el nuevo Kernel sin ningún error.	No se modificó el archivo correcto y/o hubo errores al momento de compilar el kernel.	
			Rango del punteo	25 - 15 pts	14 - 0 pts	
1.2	Implementación de la syscall getpid_counter()	30	Descripción	La variable global está correctamente declarada e implementada dentro de la función de la llamada al sistema. El código es limpio y no genera advertencias críticas.	La variable no es global (se reinicia), está mal ubicada en el flujo de la función o provoca errores de sintaxis que impiden la compilación correcta.	
			Rango del punteo	18 - 30 pts	0-17 pts	

1.3	Validación con programa de prueba	20	Descripción	El programa en espacio de usuario invoca exitosamente la llamada al sistema.	El programa de prueba falla al llamar a la función, o los logs muestran que el contador no incrementa o arroja valores aleatorios/inconsistentes.	
			Rango del punteo	20 - 12 pts	12 - 0 pts	
1.4	Uso correcto de printk y niveles de Log	5	Descripción	Se utiliza printk con un nivel de severidad adecuado para imprimir el valor del contador, facilitando su lectura en los mensajes del sistema.	No se implementó printk, se usó un nivel de severidad erróneo que oculta el mensaje, o el formato de impresión es ilegible/incorrecto.	
			Rango del punteo	5 - 3 pts	2 - 0 pts	
1.5	Uso de comando dmesg para verificación de syscall	10	Descripción	Demuestra mediante dmesg que el contador incrementa secuencialmente en cada llamada.	El contador no se visualiza mediante el comando dmesg.	
			Rango del punteo	10 - 6 pts	5 - 0 pts	
	Sub-Total de Puntos	90				
2	Conocimientos					

2.1	Informe Técnico	5	Descripción	El informe sigue la estructura solicitada, presenta capturas de pantalla claras del proceso y las conclusiones demuestran un análisis profundo sobre la diferencia entre espacios de memoria.	El informe está desordenado, carece de capturas de evidencia del proceso de compilación o las conclusiones son superficiales y no técnicas.	
			Rango del punteo	5 - 3 pts	2 - 0 pts	
2.1	Pregunta Teórica	5	Descripción	Responde la pregunta correctamente.	La respuesta es incorrecta o incompleta.	
			Rango del punteo	5 - 3 pts	2 - 0 pts	
	Sub-Total de Puntos	10				
	Total	100				

Comentarios Generales

Apartado para comentar si hubo algún inconveniente durante la calificación o comentario.
